

Shop AI: The Digital Waiter. Final Project Report (Milestone 4)

Intro to Software Engineering 2569 · School of Applied Digital Technology, MFU · Team MFU 888

***Draft for the team to review.** It reuses M1 and M2 instead of rewriting them; every number comes from a file in the repository or from `git`. What only the team can answer is marked **[team to complete]** (listed in `docs/OPEN_PLACEHOLDERS.md`).*

Members 6931503001 Aung Nyi Nyi Thant · 6931503031 Chawakron Singkaew · 6931503042 Tharathon Plengsri ·
(M1) 6931503073 Sasiwimon Chatkaew · 6931503103 Natthanon Wongsai

Repository <https://github.com/Aung-Nyi-Thant/WaiterAi> · prototype: `prototype/index.html` (no server) · app: `app/`

1. Problem and users (M1)

Small restaurants rely on a few staff to take orders and answer questions. At peak hours this means slow service, missed orders and slow answers about ingredients, allergens and hours. The people with the problem are front-of-house staff and owner-chefs; the users are diners (phone, no account), owners (laptop) and waiters and chefs (PIN). Three interviews in Chiang Rai on 22 September 2026 (M1, Gate 1):

Interviewee	Pain
May, front of house	At lunch 15–20 customers arrive at once and three staff do everything: "several tables call us at the same time... orders get missed."
Chef Kung, chef-owner	Special requests ("no fresh pork blood") are forgotten when busy; a remade bowl costs 5–7 minutes and 20–30 baht. Fears an impersonal system.
P'Nut, fried-food shop owner	The same questions all evening; a non-spicy dish arrives spicy.

Success is measured by: 90% of open questions answered within 15 s; rule-based answers within 1 s; with 3 diners and 2 staff screens for 10 minutes, no error and every reply within 30 s; a 30-question AI test with at least 90% overall and 100% on safety; never "safe", and missing allergen data never shown as "none".

2. Requirements (M2)

Full list: `docs/SRS_Shop_AI.md`; each ID is traced to code and tests in `docs/TRACEABILITY.md` (108 IDs). The M1 requirements FR-1 ... FR-10 split into 70 functional IDs: **56 Must, 14 Should**.

FR	What	Must / Should
FR-1	Multilingual (TH / MY / EN) QR menu	8 / 3
FR-2	Safe AI waiter chat	17 / 3
FR-3	Picks, orders, staff calls	7 / 0
FR-4, FR-9	Live waiter and kitchen screens; staff PIN accounts	6 / 0
FR-5, FR-6, FR-7	Owner menu and allergens, photo import, settings	16 / 4
FR-8, FR-10	QR codes per table; insights	2 / 4

NFR	Target	Measured (one laptop, <code>gemma4:12b</code> ; <code>docs/PERFORMANCE.md</code>)
NFR-P1 open questions	90% within 15 s	100 questions (EN/TH/MY): median 5.5 s , 90th percentile 10.7 s , 0 errors
NFR-P2 rule-based	within 1 s	median 17–22 ms, slowest 79 ms, with 30 diners at once
NFR-P4 capacity	3 diners + 2 staff screens, 10 min, every reply within 30 s	0 errors ; 93 AI answers, slowest 17.4 s
NFR-R1 AI unavailable	menu, picks, calls still work	fallback in 0.0 s; everything else works
NFR-P3 menu on a phone; NFR-U3 first menu in 30 min	3 s; 30 min	not measured (needs a real phone and real owners)

Acceptance criteria (happy / unhappy), examples. AI-2, AI-3: "allergic to peanuts" lists the dishes that list peanut and the dishes with no data, then says to confirm with the staff; "is the Pad Thai safe?" is never answered "safe". PC-2: sending picks creates an order with the allergy note; a sold-out dish is ignored. SF-2: five wrong PINs lock sign-in for 10 minutes. MI-3: nothing from a photo import goes live until the owner confirms.

3. Design (M2)

One Next.js 16 app (React 19, TypeScript) with SQLite and a **local** model (`gemma4:12b`, Ollama): no cloud, no paid service (charter Gate 3, SRS NFR-5). Five modules, one per team part: `diner`, `ai`, `owner`, `staff`, `platform`. **The key decision is a**

hybrid AI: everything that could hurt a customer (allergens, prices, hours, sold-out dishes, orders, the bill) is answered from the database with fixed sentences; the model only answers open questions, and each reply is checked: a safety claim, an allergen statement, a wrong price or an unknown dish is replaced by "please ask the staff" (`docs/AI_SAFETY.md`, rules R1–R14, each with a test).

Diagrams are in `docs/diagrams/` (architecture, data model, order flow, AI flow, use cases). The data model has 14 tables (`users`, `restaurants`, `menu_items` with allergens stored as a list or `NULL` = "not provided", `staff`, `orders`, `chat_messages` ...). Orders move *picked* → *new* → *cooking* → *ready* → *served*, checked by the server.

4. The prototype

`prototype/index.html` is a clickable, no-server prototype of the diner, staff and owner journeys with mock data and the real app's labels and sample menu. All **56 Must requirements** are cited by a screen, each screen has a "Covers:" bar naming its SRS IDs, and a map tab lists FR-1 ... FR-10 with an unhappy path to try for each (AI offline, wrong PIN, empty search, invalid input, flagged import rows). A test (`app/tests/prototype.test.ts`) fails if it cites an ID that is not in the SRS or misses a Must requirement.

5. Testing

Every push runs lint, type-check, unit and integration tests, the offline AI evaluation, the build, end-to-end tests against a real server (Node 22 and 24) and a Docker build. **537 unit and integration tests and 14 end-to-end tests pass; the live model scores 30/30 on the 30-question set** (once 29/30).

Scenario	Expected	Test
"Is the Pad Thai safe for a peanut allergy?"	lists allergens, asks to confirm, never "safe"	<code>ai-rules.test.ts</code>
The model says a dish is "nut-free" or a wrong price	reply replaced by "please ask the staff"	<code>ai-guard.test.ts</code>
The model is down	fallback at once; menu, picks, calls still work	<code>e2e</code>
Sold-out dish added to picks	refused	<code>orders.test.ts</code>
Chef presses "take order"; a skipped state	403; 409	<code>orders.test.ts</code>
Five wrong PINs	locked (429)	<code>srs-gaps.test.ts</code>
One owner edits another's dish	404, nothing changes	<code>auth.test.ts</code>
Text file named <code>.png</code> uploaded	rejected (first bytes checked)	<code>menu.test.ts</code>
Photo import with a missing price	row flagged; nothing live until confirmed	<code>import.test.ts</code>
Thai "call the staff"	calls staff, not read as a sesame allergy	<code>thai-keywords.test.ts</code>

Not covered by automated tests: how screens look on real phones, the Thai and Burmese wording of labels and new allergen words (needs a native speaker), page load on a phone over Wi-Fi, real owners, photo import on real menus.

6. Golden Thread (problem → requirement → design → feature → test)

Pain	Requirement	Design and feature	Test
Same questions all day	FR-2 / AI-5, AI-9	rules + checked local model; Ask AI	<code>ai-rules</code> , 30-question eval
Allergy and special requests forgotten	AI-2, AI-3, PC-2, PC-7	allergen data only from the database; allergy note travels with the order; banner on staff screens	<code>allergy-profile</code> , <code>ai-guard</code>
Tables call at once; orders missed	PC-3, SF-3, SF-4	live screens (3 s), one open call per table	<code>orders</code> , <code>e2e</code>
New staff do not know the menu	DM-1...7, MM-1, MM-2	QR menu in 3 languages; owner keeps allergens complete	<code>menu</code> , <code>diner-filters</code>
Keeping a menu current	MI-1...4, QR-1	photo import with review; QR per table	<code>import</code>
Unseen unmet demand	IN-1...4	questions logged without personal data	<code>chat-api</code> , <code>owner-insights</code>

7. AI-first reflection

Full log: `docs/AI_USAGE_LOG.md`; statement: `docs/AI_USE_STATEMENT.md`. The product's model `gemma4:12b` is part of the system; Claude Code (Sonnet 5 and 5.5) was the development tool. **[team to complete: other tools, and which member used which]**

What worked. Tests written first found real bugs (a "bill" keyword matching inside "papaya"; a wrong quantity in "x4 ... x2"; the Thai word for "staff" read as "sesame"). Measuring beat guessing: the slow first answer after idle was the long prompt being read (40–65 s), not the model loading; a start-up warm-up now covers it.

What people decided. The "AI" versus "staff" wording (corrected twice by a member), the Burmese replies (four native-

speaker reviews), no cloud AI, and how to merge two lines of AI-assisted work. **[team to complete: what each member changed or rejected in the AI's output]**

Lessons. A language model must never be the source of a safety fact. A claim is worth its test: we reverted fixes to see the tests fail, and one test passed even with the bug. Numbers need an idle machine: a first load test "passed" while 104 of 108 answers were the fallback message, and was re-run.

8. Team and individual contribution

Parts (`docs/TEAM_WORK_PLAN.md`): 1 diner app · 2 AI waiter and backend core · 3 owner back office · 4 staff and orders · 5 QR, insights, import screen, tests, docs. Accounts: `Tharathon47` = Tharathon Plengsri (6931503042, part 1) · `Aung-Nyi-Thant` = Aung Nyi Nyi Thant (6931503001, part 2) · `chawakron50-lab` = Chawakron Singkaew (6931503031, part 3) · `wujieiei` = Natthanon Wongsai (6931503103, part 4) · `noeysasi` = Sasiwimon Chatkaew (6931503073, part 5).

What GitHub shows today (21 Sep – 6 Oct 2026): all commits on `main` and all pull requests come from the account `Aung-Nyi-Thant`, so the other members' work is not yet visible in the repository. **[team to complete: for each member, their commits, pull requests, documents, diagrams, tests or reviews in the repository, and where non-code work lives]**

9. Limits

Speed figures are from one laptop and a 13-dish menu (a longer menu means a slower first answer). With three diners chatting nonstop about 1 answer in 7 is the fallback message, because they wait for one model. The first question after a restart or a menu edit can fall back for up to a minute. Not yet done: tests on real phones and with real owners, native review of all Thai and Burmese labels, a public HTTPS deployment.